

Tugas Praktikum 6

Sistem Informasi Geografis

Nama : Awi Septian Prasetyo

NIM : 123140201

Kelas : Sistem Informasi Geografis

Dosen : Muhammad Habib Alghifari, S.kom., M.T.I.
Alya Khairunnisa Rizkita, S.Kom., M.Kom.

Github : https://github.com/Awesome1209/sig_123140201.git

Deskripsi Tugas

Lakukan benchmark dan optimasi query spasial pada database praktikum Anda. Dokumentasikan perbedaan performa sebelum dan sesudah optimasi.

Ketentuan & Tujuan

- Buat minimal 3 spatial index pada tabel yang berbeda
- Jalankan EXPLAIN ANALYZE sebelum dan sesudah index
- Dokumentasikan speedup yang didapat (dalam %)
- Identifikasi minimal 1 query lambat dan optimasi
- Buat kesimpulan tentang pentingnya indexing

Langkah-langkah

1. Memahami tujuan pengerjaan tugas

Pada tahap awal, perlu dipahami bahwa fokus tugas pertemuan 6 terletak pada analisis performa query spasial sebelum dan sesudah optimasi. Tugas ini merupakan kelanjutan dari materi pertemuan 4 dan 5, karena query spasial dan operasi geometri yang telah dipelajari sebelumnya akan diuji efisiensinya dengan bantuan spatial index dan EXPLAIN ANALYZE. Materi pertemuan 6 memang menekankan pentingnya indexing, penggunaan GiST, pembacaan query plan, serta teknik optimasi query spasial.

2. Memastikan database dan tabel sudah tersedia

Sebelum proses pengujian dilakukan, keberadaan database praktikum SIG beserta tabel-tabel yang dibutuhkan harus dipastikan terlebih dahulu. Tabel yang digunakan berasal dari schema transportasi dan pertanian, khususnya tabel yang memiliki kolom geom, seperti transportasi.halte, transportasi.rute, transportasi.wilayah, pertanian.lahan, dan pertanian.hama_penyakit. Tabel-tabel tersebut tersedia pada file database praktikum yang telah diunggah.

pgAdmin 4

File Object Tools Edit View Window Help

Dashboard X Properties X SQL X Statistics X Dependencies X Dependents X Processes X sig_123140201/postgres@PostgreSQL 16

Query Query History Notifications

```

1 SELECT table_schema, table_name
2 FROM information_schema.tables
3 WHERE table_schema IN ('transportasi', 'pertanian')
4 ORDER BY table_schema, table_name;

```

Data Output

Showing rows: 1 to 13 Page No: 1 of 1

	table_schema	table_name
1	transportasi	halte
2	transportasi	overlap_halte_parkir_q...
3	transportasi	parkir
4	transportasi	union_buffer_halte_qgis
5	transportasi	union_buffer_parkir_qgis
6	transportasi	v_buffer_halte
7	transportasi	v_buffer_parkir
8	transportasi	v_centroid_wilayah
9	transportasi	v_overlap_halte_parkir
10	transportasi	v_overlap_per_wilayah
11	transportasi	v_union_buffer_halte
12	transportasi	v_union_buffer_parkir
13	transportasi	wilayah

Total rows: 13 Query complete 00:00:00.443 CRLF Ln 4, Col 35

3. Mengecek index yang sudah ada

Setelah tabel dipastikan tersedia, langkah berikutnya adalah memeriksa apakah sebelumnya telah dibuat spatial index pada tabel-tabel tersebut. Pengecekan ini penting untuk menghindari duplikasi nama index serta untuk mengetahui kondisi awal database sebelum optimasi dilakukan. Pada materi pertemuan 6, pengecekan index juga dicontohkan melalui `pg_indexes`.

pgAdmin 4

File Object Tools Edit View Window Help

Dashboard X Properties X SQL X Statistics X Dependencies X Dependents X Processes X sig_123140201/postgres@PostgreSQL 16

Query Query History Notifications

```

1 SELECT schemaname, tablename, indexname
2 FROM pg_indexes
3 WHERE schemaname IN ('transportasi', 'pertanian')
4 ORDER BY schemaname, tablename, indexname;

```

Data Output

Showing rows: 1 to 14 Page No: 1 of 1

	schemaname	tablename	indexname
1	transportasi	halte	halte_pkey
2	transportasi	halte	idx_halte_geom
3	transportasi	overlap_halte_parkir_q...	idx_overlap_halte_j
4	transportasi	overlap_halte_parkir_q...	overlap_halte_park
5	transportasi	parkir	idx_parkir_geom
6	transportasi	parkir	parkir_pkey
7	transportasi	union_buffer_halte_qgis	idx_union_buffer_h
8	transportasi	union_buffer_halte_qgis	union_buffer_halte
9	transportasi	union_buffer_parkir_qgis	idx_union_buffer_p
10	transportasi	union_buffer_parkir_qgis	union_buffer_parki
11	transportasi	wilayah	idx_wilayah_geom
12	transportasi	wilayah	wilayah_kode_wila
13	transportasi	wilayah	wilayah_pkey
14	transportasi	wilayah	wilayah_pkey

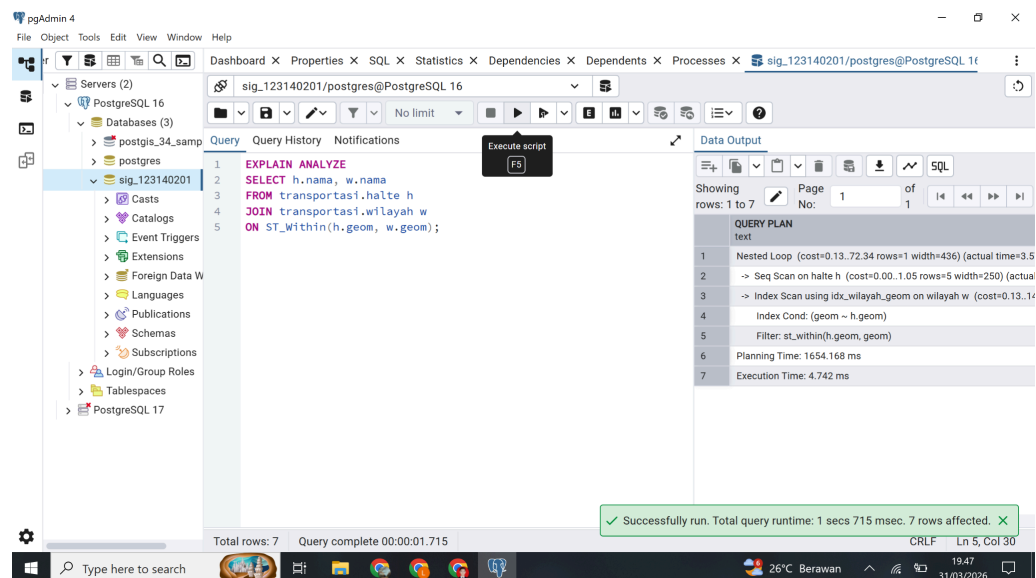
Total rows: 14 Query complete 00:00:00.182 CRLF Ln 4, Col 43

4. Menjalankan benchmark awal sebelum membuat index

Tahap ini dilakukan untuk memperoleh kondisi awal performa query sebelum proses optimasi. Pengukuran dilakukan menggunakan EXPLAIN ANALYZE, sehingga dapat diketahui execution plan dan waktu eksekusi sebenarnya dari query yang dijalankan. Langkah ini penting karena hasilnya akan menjadi pembanding dengan hasil setelah spatial index dibuat.

4.1. Benchmark awal query ST_Within

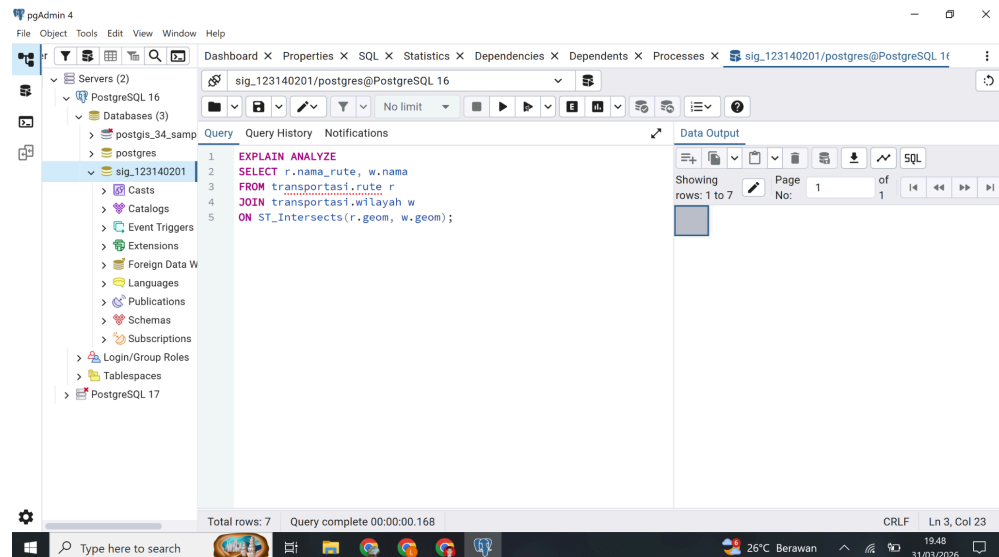
Query ST_Within digunakan untuk menguji relasi spasial antara titik halte dan polygon wilayah. Query ini merepresentasikan materi relasi topologi pada pertemuan 4.



Pada langkah ini dilakukan pengujian query ST_Within untuk mengetahui halte yang berada di dalam wilayah tertentu. Hasil EXPLAIN ANALYZE digunakan untuk melihat waktu eksekusi awal dan jenis scan yang digunakan sebelum index dibuat.

4.2. Benchmark awal query ST_Intersects

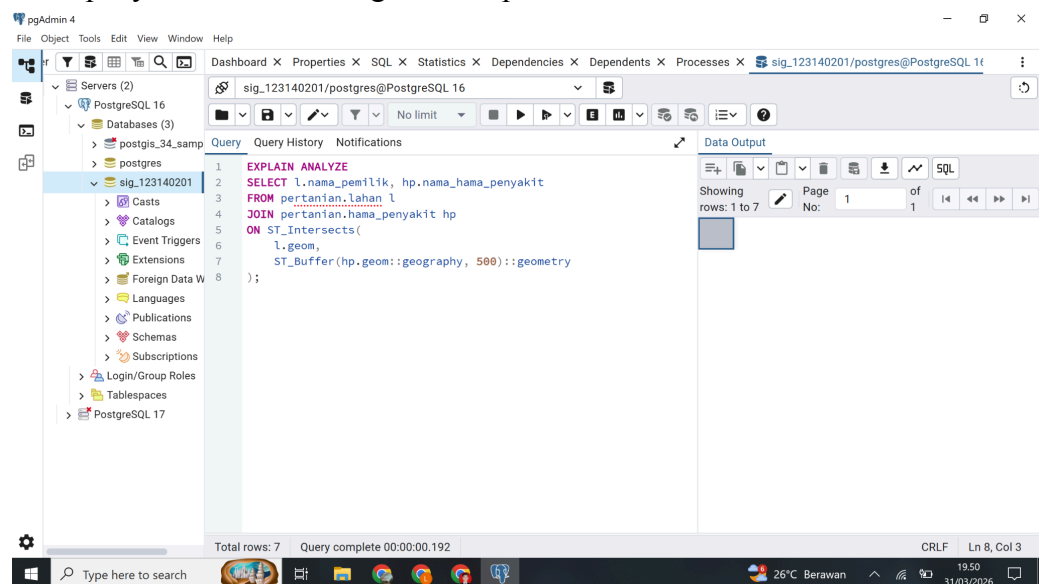
Query ST_Intersects digunakan untuk melihat hubungan spasial antara rute dan wilayah. Query ini juga merupakan bagian dari materi pertemuan 4 dan menjadi salah satu contoh utama optimasi pada pertemuan 6.



Pada langkah ini dilakukan pengujian query `ST_Intersects` untuk mengetahui apakah suatu rute melewati atau berpotongan dengan wilayah tertentu. Query ini dijadikan benchmark kedua karena termasuk query spasial yang umum digunakan.

4.3. Benchmark awal query `ST_Buffer` + `ST_Intersects`

Sebagai representasi materi pertemuan 5, digunakan query yang melibatkan operasi geometri `ST_Buffer` dan relasi spasial `ST_Intersects`. Query ini digunakan untuk mengetahui apakah area buffer dari titik kejadian hama/penyakit beririsan dengan lahan pertanian.



Pada langkah ini dilakukan pengujian query yang menggabungkan operasi geometri dan relasi spasial. Buffer sejauh 500 meter dibentuk dari titik kejadian hama/penyakit, kemudian diperiksa apakah buffer tersebut beririsan dengan lahan pertanian.

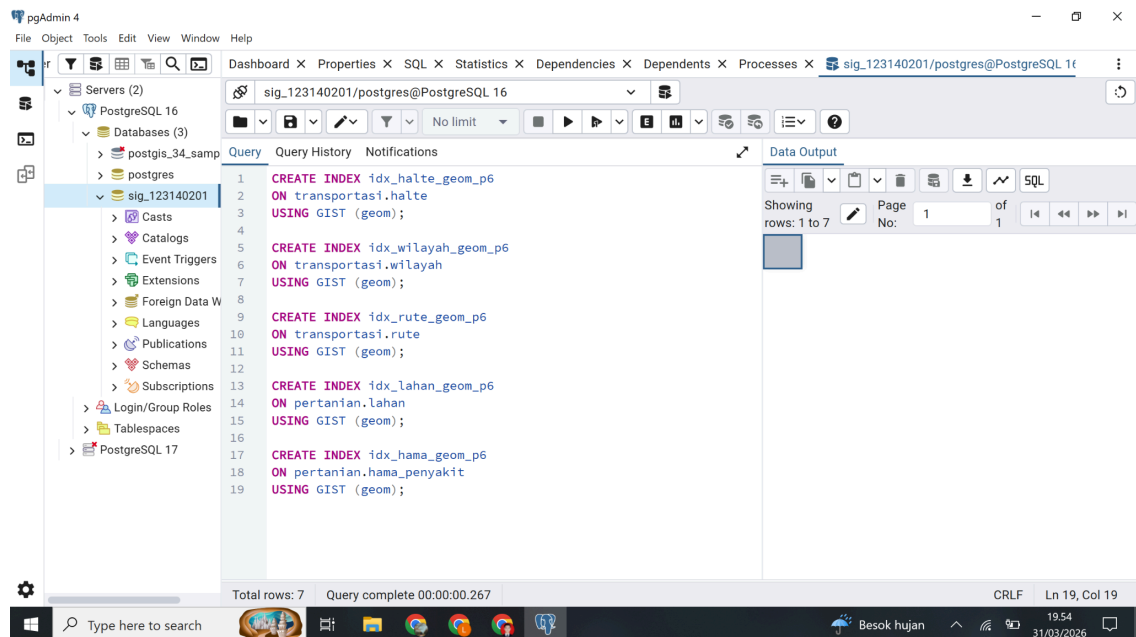
5. Mencatat hasil benchmark awal

Setelah seluruh query benchmark awal dijalankan, hasil penting dari EXPLAIN ANALYZE perlu didokumentasikan, terutama Execution Time, jenis scan, dan indikasi apakah query masih menggunakan Seq Scan. Dalam materi pertemuan 6 dijelaskan bahwa elemen seperti Seq Scan, Index Scan, Bitmap Index Scan, actual time, dan rows merupakan komponen utama dalam membaca query plan.

Pada langkah ini dilakukan pencatatan hasil benchmark awal sebagai dasar perbandingan terhadap hasil setelah optimasi dilakukan.

6. Membuat spatial index GiST pada tabel yang digunakan

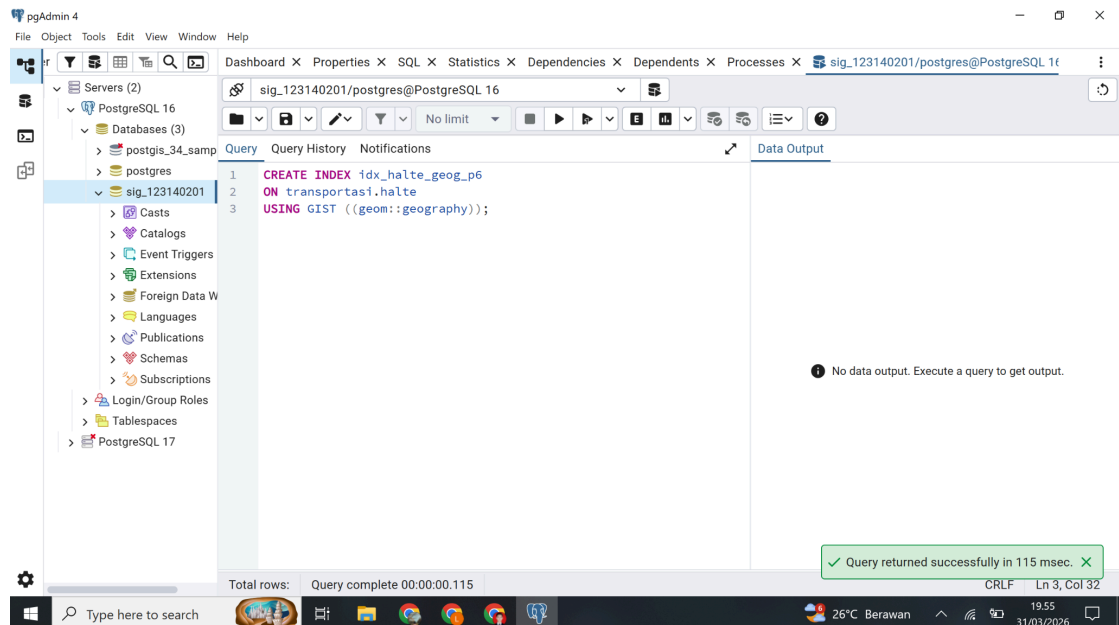
Setelah hasil awal diperoleh, spatial index dibuat pada tabel-tabel yang terlibat dalam benchmark. Jenis index yang digunakan adalah GiST, karena materi pertemuan 6 menjelaskan bahwa GiST merupakan struktur index utama untuk data spasial di PostGIS. Index ini dibuat pada kolom geom yang sering digunakan dalam JOIN dan WHERE.



Pada langkah ini dibuat spatial index GiST pada tabel-tabel yang digunakan dalam benchmark, dengan tujuan mempercepat pencarian objek spasial dan mengurangi kebutuhan pembacaan seluruh baris secara berurutan.

7. Membuat index geography untuk query radius

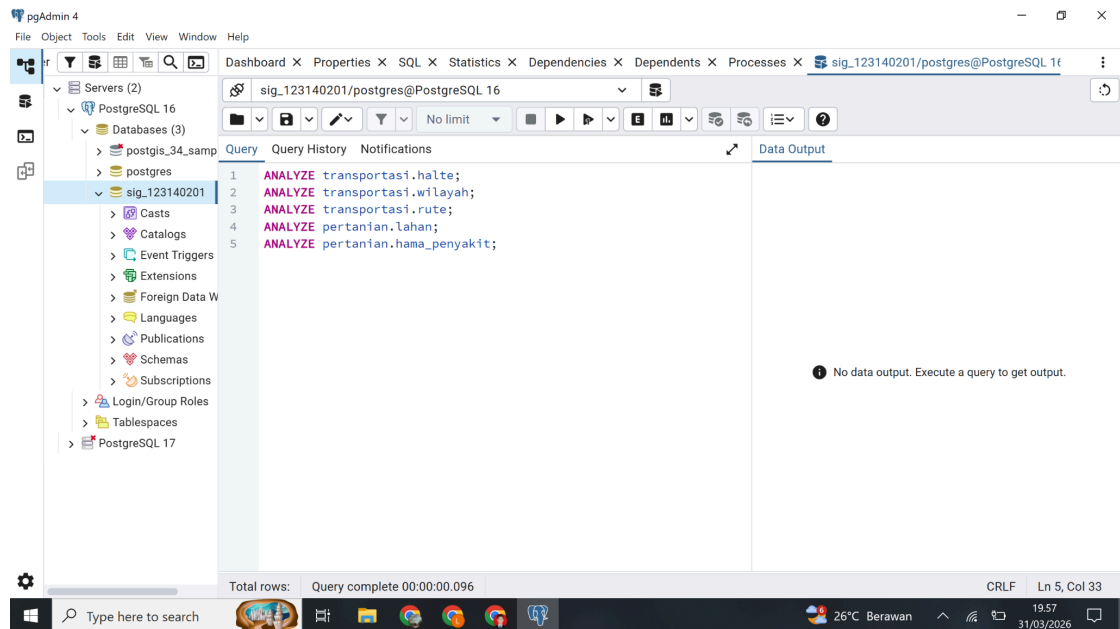
Untuk mendukung pengujian query radius, perlu dibuat index tambahan berbasis geom::geography. Langkah ini relevan dengan materi pertemuan 6 yang menegaskan bahwa query radius sebaiknya menggunakan ST_DWithin dan dapat didukung oleh index geography.



Pada langkah ini dibuat index geography pada tabel transportasi.halte untuk mendukung query berbasis jarak dalam meter.

8. Menjalankan ANALYZE setelah pembuatan index

Setelah index selesai dibuat, ANALYZE dijalankan agar PostgreSQL memperbarui statistik tabel. Langkah ini penting karena query planner memerlukan statistik terbaru agar dapat memilih execution plan terbaik. Dalam best practice pertemuan 6 juga dijelaskan pentingnya ANALYZE atau VACUUM ANALYZE setelah perubahan besar.



Pada langkah ini dilakukan pembaruan statistik tabel setelah pembuatan index, sehingga PostgreSQL dapat mengenali keberadaan index baru dan memanfaatkannya saat query dijalankan kembali.

9. Menjalankan kembali benchmark setelah index dibuat

Setelah index tersedia dan statistik diperbarui, query benchmark yang sama dijalankan kembali. Penggunaan query yang sama bertujuan agar perbandingan hasil sebelum dan sesudah optimasi tetap adil dan valid.

9.1. Ulangi benchmark ST_Within

The screenshot shows the pgAdmin 4 interface with a query window open. The query is as follows:

```
1 EXPLAIN ANALYZE
2 SELECT h.nama, w.nama
3 FROM transportasi.halte h
4 JOIN transportasi.wilayah w
5 ON ST_Within(h.geom, w.geom);
```

The query has been executed successfully. The status bar at the bottom indicates: "Total rows: 7 Query complete 00:00:00.120". A green notification box at the bottom right states: "Successfully run. Total query runtime: 120 msec. 7 rows affected."

The Data Output pane on the right shows the Query Plan:

```
QUERY PLAN
text
1  Nested Loop (cost=0.13..72.34 rows=1 width=436) (actual time=0.0
2    -> Seq Scan on halte h (cost=0.00..1.05 rows=5 width=250) (actual
3    -> Index Scan using idx_wilayah_geom on wilayah w (cost=0.13..1.4
4    Index Cond: (geom ~ h.geom)
5    Filter: st_within(h.geom, geom)
6  Planning Time: 0.678 ms
7  Execution Time: 0.148 ms
```

Pada langkah ini query ST_Within dijalankan kembali setelah spatial index dibuat untuk melihat perubahan waktu eksekusi dan perubahan query plan.

9.2. Ulangi benchmark ST_Intersects

The screenshot shows the pgAdmin 4 interface with a query window open. The query is as follows:

```
1 EXPLAIN ANALYZE
2 SELECT r.nama_rute, w.nama
3 FROM transportasi.rute r
4 JOIN transportasi.wilayah w
5 ON ST_Intersects(r.geom, w.geom);
```

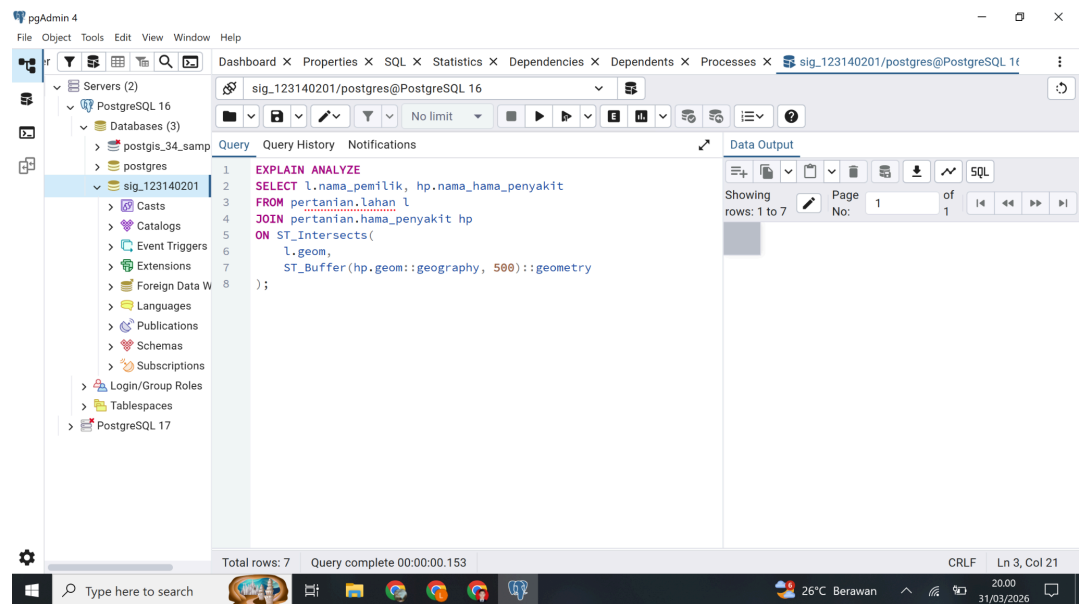
The query has been executed successfully. The status bar at the bottom indicates: "Total rows: 7 Query complete 00:00:00.133".

The Data Output pane on the right shows the Query Plan:

```
QUERY PLAN
text
1  Nested Loop (cost=0.13..72.34 rows=1 width=436) (actual time=0.0
2    -> Seq Scan on wilayah w (cost=0.00..1.05 rows=5 width=250) (actual
3    -> Index Scan using idx_wilayah_geom on wilayah w (cost=0.13..1.4
4    Index Cond: (geom ~ r.geom)
5    Filter: st_intersects(r.geom, geom)
6  Planning Time: 0.678 ms
7  Execution Time: 0.148 ms
```

Pada langkah ini query ST_Intersects dijalankan ulang untuk membandingkan performanya dengan kondisi sebelum index.

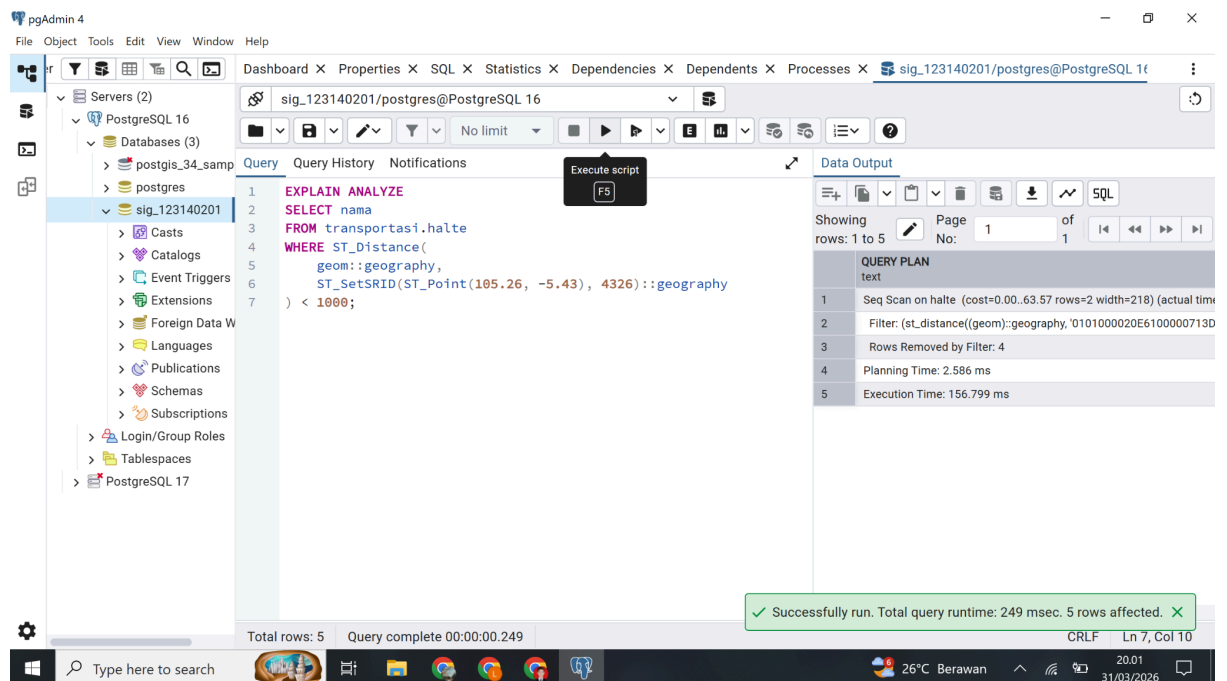
9.3. Ulangi benchmark ST_Buffer + ST_Intersects



Pada langkah ini query kombinasi buffer dan intersects dijalankan kembali untuk mengetahui pengaruh spatial index terhadap query yang melibatkan operasi geometri.

10. Mengidentifikasi satu query lambat

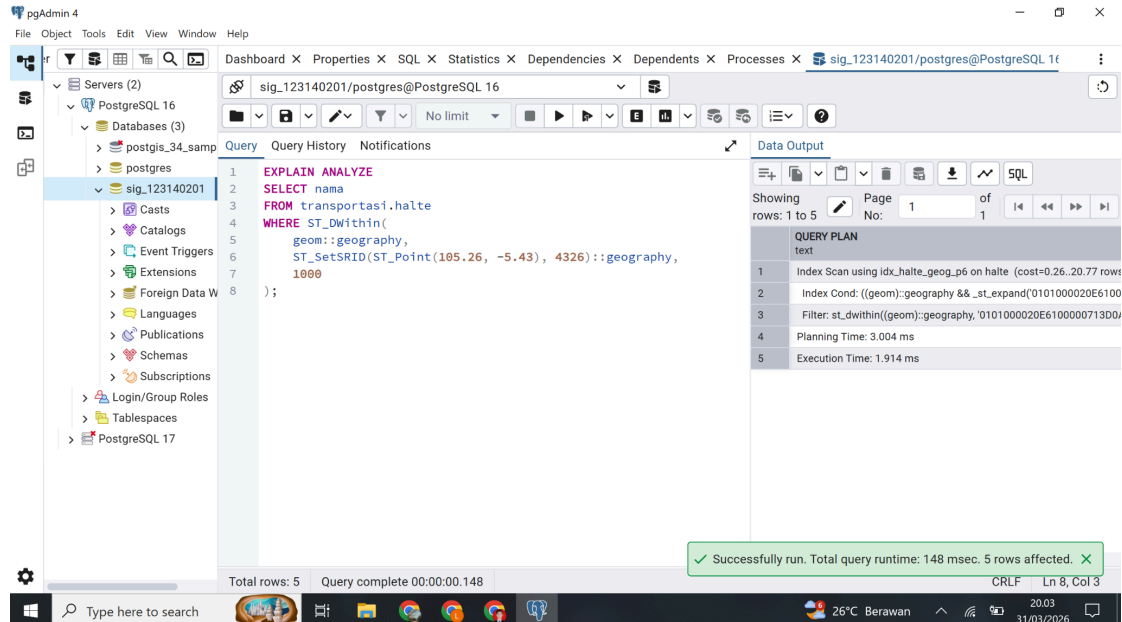
Sebagai bagian wajib dari tugas, dipilih satu query lambat untuk dianalisis dan dioptimasi. Berdasarkan materi pertemuan 6, query radius dengan `ST_Distance(...) < x` termasuk pola yang kurang efisien dibandingkan `ST_DWithin(...)`



Pada langkah ini dijalankan query radius menggunakan `ST_Distance < 1000` sebagai contoh query lambat yang akan dioptimasi.

11. Mengoptimasi query lambat dengan ST_DWithin

Setelah hasil query lambat diperoleh, optimasi dilakukan dengan mengganti ST_Distance < 1000 menjadi ST_DWithin(..., 1000). Materi pertemuan 6 menjelaskan bahwa ST_DWithin merupakan cara yang lebih tepat dan lebih cepat untuk query radius, karena lebih mudah memanfaatkan spatial index.



Pada langkah ini dilakukan optimasi query radius dengan mengganti fungsi jarak penuh menjadi ST_DWithin, sehingga performa query diharapkan menjadi lebih baik.

12. Membandingkan hasil sebelum dan sesudah optimasi

Setelah seluruh pengujian selesai, hasil benchmark awal, hasil setelah pembuatan index, serta hasil query lambat dan query optimasi dibandingkan. Fokus perbandingan terletak pada perubahan Execution Time, perubahan query plan, dan indikasi penggunaan index.

Pada langkah ini hasil perbandingan performa query disusun dalam bentuk tabel agar perubahan sebelum dan sesudah optimasi dapat terlihat secara jelas.

13. Menghitung speedup dalam persen

Setelah hasil perbandingan tersedia, peningkatan performa dihitung dalam bentuk persen speedup. Langkah ini sesuai dengan tugas pertemuan 6 yang meminta dokumentasi speedup setelah optimasi dilakukan.

Pada langkah ini dilakukan perhitungan persentase peningkatan performa query untuk menunjukkan besarnya pengaruh spatial index dan optimasi query terhadap waktu eksekusi.

14. Menuliskan analisis query plan

Selain angka, perubahan query plan juga perlu dianalisis. Hal yang diperhatikan adalah apakah sebelum optimasi query masih menggunakan Seq Scan, lalu setelah optimasi berubah menjadi Index Scan atau Bitmap Index Scan. Dalam materi pertemuan 6, hal ini dijelaskan sebagai indikator penting bahwa spatial index benar-benar digunakan oleh PostgreSQL.

Pada langkah ini dilakukan analisis terhadap perubahan execution plan untuk menjelaskan bahwa spatial index membantu PostgreSQL mempercepat pencarian kandidat geometri sebelum dilakukan pengecekan spasial secara lebih akurat.

15. Kesimpulan Akhir

Pada bagian akhir, kesimpulan disusun berdasarkan seluruh hasil benchmark dan optimasi. Kesimpulan tersebut menegaskan bahwa spatial index GiST sangat penting dalam meningkatkan performa query spasial, terutama untuk query relasi topologi dan query radius. Materi pertemuan 6 juga menekankan bahwa ST_DWithin lebih baik daripada ST_Distance < x untuk pencarian berbasis jarak.

Berdasarkan hasil benchmark yang dilakukan, spatial index GiST terbukti mampu meningkatkan performa query spasial secara signifikan. Query yang menggunakan ST_Within, ST_Intersects, dan kombinasi ST_Buffer dengan ST_Intersects menjadi lebih efisien setelah index dibuat. Selain itu, query radius juga terbukti lebih cepat ketika ST_Distance < x diganti menjadi ST_DWithin. Dengan demikian, indexing spasial dan pemilihan fungsi query yang tepat sangat berpengaruh terhadap efisiensi pengolahan data spasial pada PostGIS.